

```
package com.ravi.predicate;

import java.util.function.Predicate;

record Student(Integer id, String name, Double marks)
{
}

public class PredicateDemo3
{
    public static void main(String[] args)
    {
        //Verify whether a student is pass or fail

        Student []students = new Student[4]; //Student Array

        students[0] = new Student(111, "Scott", 75D);
        students[1] = new Student(222, "Alen", 45D);
        students[2] = new Student(333, "John", 65D);
        students[3] = new Student(444, "Smith", 35D);

        Predicate<Student> p4 = std -> std.marks() >=50;

        for(Student student : students)
        {
            if(p4.test(student))
            {
                IO.println(student.name()+" is passed in the exam ");
            }
            else
            {
                IO.println(student.name()+" is not passed in the exam ");
                IO.println("Try Again");
            }
        }
    }
}
```

```
package com.ravi.predicate;

import java.util.function.Predicate;

/* Verify whether a Car is eligible for re-sale or Not : Criteria
 * a) Price must be <= 500000
 * b) Mileage must be >= 18
 * c) Model >=2021
 */

record Car(Integer model, String name, Double price, Integer mileage)
{
}

public class PredicateDemo4 {

    public static void main(String[] args)
    {
        Predicate<Car> p4 = car -> car.price() <= 500000 && car.mileage() >= 18 &&
car.model() >=2021;

        Car userCar = new Car(2022, "Swift", 450000D, 18);

        boolean isReadyForSale = p4.test(userCar);

        if(isReadyForSale)
        {
            IO.println(userCar.name()+" is ready for re-sale");
        }
        else
        {
            IO.println("Not expected as per customer requirement");
        }
    }
}
```

Consumer<T> Functional Interface :

* It is a predefined functional interface available in java.util.function sub package

```
@FunctionalInterface
public interface Consumer<T>
{
    void accept(T x);
}
```

* It is used to consume the value, It has an abstract method accept(T x) which takes single parameter T and does not return any value.

```
//Program :
package com.ravi.consumer;

import java.util.function.Consumer;

record Product(Integer id, String name, Double price)
{
}

public class ConsumerDemo
{
    public static void main(String[] args)
    {
        Consumer<Product> c1 = prod ->
        {
            IO.println("Product Id is :"+prod.id());
            IO.println("Product Name is :"+prod.name());
            IO.println("Product Price is :"+prod.price());
        };
        c1.accept(new Product(111, "Laptop", 95000D));
    }
}
```

```
package com.ravi.consumer;

import java.util.function.Consumer;

public class ConsumerDemo
{
    public static void main(String[] args)
    {
        Consumer<Integer> square = num -> IO.println("Square is :"+(num*num));
        Consumer<String> upper = str -> IO.println("Upper Case :"+str.toUpperCase());

        square.accept(5);
        upper.accept("Java");
    }
}
```

Function<T,R> Functional Interface :

* It is a predefined functional interface available in java.util.function sub package

```
@FunctionalInterface
public interface Function<T,R>
{
    R apply(T x);
}
```

* Type Parameters:
T - the type of the INPUT to the function.
R - the type of the RESULT of the function. (return type)

* It provides an abstract method apply that accepts one argument(T) and produces a result(R).

* Note :- The type of T(input) and the type of R(Result) both will be decided by the developer.

```
package com.ravi.function_demo;

import java.util.function.Function;

public class FunctionDemo1
{
    public static void main(String[] args)
    {
        //Find the length of the given city

        Function<String,Integer> fn1 = city -> city.length();

        String cityName = IO.readLine("Enter the name of the city :");

        IO.println("The length of "+cityName+" is :"+fn1.apply(cityName));
    }
}
```

WAP to return the salary and bonus amount of an Employee

```
package com.ravi.function_demo;

import java.util.function.Function;

record Employee(Integer id, String name, Double salary)
{
}

public class FunctionDemo2
{
    public static void main(String[] args)
    {
        Function<Employee,Double> fn2 = emp ->
        {
            double bonus = 0.0;
            if(emp.salary()>=100000)
            {
                bonus = emp.salary() * 0.20;
            }
            else
            {
                bonus = emp.salary() * 0.10;
            }

            return emp.salary() + bonus;
        };

        Employee e1 = new Employee(111, "Scott", 90000D);
        IO.println(e1.name()+" Salary + bonus amount is :"+fn2.apply(e1));
    }
}
```

Supplier<T>

* It is a predefined functional interface available in java.util.function sub package.

```
@FunctionalInterface
public interface Supplier<T>
{
    T get();
}
```

* It provides an abstract method get() which does not take any argument but produces/supply/return a value of type T.

```
//Program :
package com.ravi.supplier;

import java.util.function.Supplier;

public class SupplierDemo1 {

    public static void main(String[] args)
    {
        Supplier<String> s1 = () -> 12+90+"Java"+56+90;
        IO.println("Value is :"+s1.get());
    }
}
```

```
package com.ravi.supplier;

import java.util.function.Supplier;

record Player(Integer id, String name, Double basePrice)
{
}

public class SupplierDemo2 {

    public static void main(String[] args)
    {
        Supplier<Player> s2 = () ->
        {
            Integer id = Integer.parseInt(IO.readLine("Enter Player id :"));
            String name = IO.readLine("Enter Player Name :");
            Double price = Double.parseDouble(IO.readLine("Enter Player Price :"));
            return new Player(id, name, price);
        };

        Player player = s2.get();
        IO.println(player);
    }
}
```

How to generate a random number based on the user choice ?

* There is predefined class called java.util.**Random** through which we can generate a random number based on the user provided limit.

* It provides a non static metod **nextInt(int bounds)** through which generate a random number based on the given bound where bound value will be exclusive.

* We cannot pass the bound value 0 OR -ve otherwise It will generate an exception **IllegalArgumentException**

```
package com.ravi.supplier;

import java.util.Random;

public class RandomNumberGenerator {

    public static void main(String[] args)
    {
        Random r = new Random();
        int num = r.nextInt(100); //0 to 99
        IO.println(num);
    }
}
```